Detec Logo

Detec Collector Engine: Technical Report

Document type: Security and DevOps technical report
Scope: Telemetry collection, behavioral signal analysis, confidence scoring, evasion detection, policy enforcement
Audience: Security engineers, DevOps, platform owners
Version: 1.0
Playbook alignment: v0.4.0 (rules, confidence formula, detection profiles)

Executive Summary

The Detec endpoint agent (collector) implements a multi-stage pipeline: **telemetry collection** into a thread-safe event store, **tool-specific and behavioral scanning** that produce layer signals, **confidence scoring** via a weighted five-layer model with penalties and evasion boost, **evasion detection** as a cross-cutting scanner (Playbook Rule C6), and **policy evaluation** with deterministic escalation and optional network/container overlays. Enforcement is posture-driven (passive / audit / active) and rate-limited, with process kill, network block, and resurrection escalation. This report describes each component in enough detail for security review and operational tuning.

---

1. Telemetry Collection Layer

1.1 Purpose and Design

The telemetry layer decouples **event acquisition** from **consumption by scanners**. Providers push raw events into a central store; scanners read from the store during each scan cycle. This allows:

- Multiple providers (polling, native OS hooks) to be swapped or composed.
- Event-driven alerts (e.g., out-of-cycle scan when a known agentic process starts).
- Consistent retention and eviction so scanners see a bounded, time-limited window of data.

1.2 Event Types and Schema

Three typed event classes live in `collector/telemetry/event_store.py`:

Event class	Fields (summary)	Use
`ProcessExecEvent`	`timestamp`, `pid`, `ppid`, `name`, `cmdline`, `username`, `binary_path`, `source`	Process tree and name-based detection
`NetworkConnectEvent`	`timestamp`, `pid`, `process_name`, `remote_addr`, `remote_port`, `local_port`, `protocol`, `sni`, `source`	Network layer and LLM cadence
`FileChangeEvent`	`timestamp`, `path`, `action` (created/modified/deleted/renamed), `pid`, `process_name`, `source`	File-layer and burst-write behavior

Each event carries a `source` string (e.g. `"polling"`, `"esf"`, `"etw"`, `"ebpf"`) for attribution and for heuristics that depend on real-time vs. polled data.

1.3 Event Store (Ring Buffer)

`EventStore` is a **thread-safe ring buffer** with:

- **Per-type deques:** `_process_events`, `_network_events`, `_file_events`, each with configurable `maxlen` (default 10,000).
- **Retention:** Configurable `retention_seconds` (default 120). On each `get_*` call, events older than `now − retention_seconds` are evicted; only events within the window are returned.
- **Lock:** A single `threading.Lock` guards all pushes and eviction/read logic so that background provider threads and the scan thread do not race.

Write API:

- `push_process(event)`
- `push_network(event)`
- `push_file(event)`

Read API (used by scanners):

- `get_process_events(name_pattern=None, since=None)`
- `get_network_events(pid=None, remote_addr=None, since=None)`
- `get_file_events(path_prefix=None, since=None)`

Alert callback: `push_process` can invoke an optional `on_alert(event)` callback **outside the lock** when the event matches a fast heuristic (see Section 1.5). The daemon uses this to trigger an immediate out-of-cycle scan.

1.4 Telemetry Providers

Interface (`collector/providers/base.py`): `TelemetryProvider` defines `name`, `available()`, `start(store)`, and `stop()`.

Registry (`collector/providers/registry.py`):

- **Preference `"auto"`:** Try platform-native provider first; on failure or unsupported platform, fall back to polling.
- **Preference `"native"`:** Use only native; raise if unavailable.
- **Preference `"polling"`:** Use only the polling provider.

Platform mapping:

- **macOS:** Endpoint Security Framework (ESF) via `ESFProvider` (when available).
- **Linux:** eBPF-based provider (`EBPFProvider`) when available.
- **Windows:** ETW-based provider (`ETWProvider`) when available.

Polling provider (`collector/providers/polling.py`):

- **Process:** Iterates `psutil.process_iter(["pid", "name", "cmdline", "username", "ppid"])` and pushes one `ProcessExecEvent` per process with `source="polling"`.
- **Network:** Uses `psutil.net_connections(kind="tcp")`; for each connection with `laddr`/`raddr`, pushes a `NetworkConnectEvent` (process name resolved from `conn.pid` when possible).

- **File:** Not populated by the polling provider (file events require native hooks or future extensions).
- **Invocation:** The provider does not run a background thread; the main scan loop calls `provider.poll()` once per cycle before running scanners, so the event store is filled on demand in one-shot or daemon mode.

Operational note: If a native provider fails to start in daemon mode, the code falls back to `PollingProvider` and continues; only if the provider is explicitly `"native"` does startup fail when native is unavailable.

1.5 Out-of-Cycle Alert Heuristic

To support event-driven scans when a native provider is used, `EventStore._should_alert(event)` is called from `push_process` (under the lock). It returns true if either:

1. **Agentic process name:** The process name (or cmdline) contains one of a fixed set of patterns (e.g. `claude`, `cursor`, `ollama`, `copilot`, `aider`, `interpreter`, `openclaw`, `continue`, `gpt-pilot`, `lm-studio`, `cline`, `codex`, `devin`, and others).
2. **Shell fan-out:** The process is a known shell (`bash`, `sh`, `zsh`, etc.), and the same parent PID has already spawned at least five shell children within the retention window (per-ppid counter in `_shell_children_by_ppid`).

When true, `on_alert(event)` is invoked so the daemon can schedule an immediate scan without waiting for the next interval.

1.6 Security and Operational Considerations

- **Permissions:** Polling requires read access to process list and network connections; on some systems this may require elevated or specific capabilities. Native providers (ESF, ETW, eBPF) have their own permission and deployment requirements.
- **Data volume:** Ring buffer and retention limit memory and CPU; eviction is lazy on read. Tuning `max_events` and `retention_seconds` may be needed for very busy endpoints.
- **No persistence:** The store is in-memory only; process/network/file history is lost across agent restarts.

---

2. Behavioral Signal Analysis

2.1 Role in the Pipeline

Behavioral analysis serves two purposes:

1. **Named-tool scanners** (Claude Code, Cursor, Ollama, etc.) produce a `ScanResult` with a **five-layer signal vector** (`process`, `file`, `network`, `identity`, `behavior`) and optional penalties. Those signals feed the confidence model (Section 3).
2. A **BehavioralScanner** runs after all named scanners and detects **unknown agentic entities** by pattern (BEH-001 through BEH-008) without relying on tool name. It sets `tool_name = "Unknown Agent"` and `tool_class = "C"` (or `"D"` when BEH-008 resurrection is matched).

Behavioral signals are thus both a **layer** in the confidence formula and the **basis for pattern-based detection** when no named scanner matches.

2.2 Layer Signals and ScanResult

Every scanner returns a `ScanResult` (`collector/scanner/base.py`) with:

- **`signals: LayerSignals`**
  Five floats in [0, 1]: `process`, `file`, `network`, `identity`, `behavior`.
  Indicate strength of evidence in each dimension (e.g. process running, config files present, LLM connections, user/session identity, behavioral patterns).
- **`penalties: list[tuple[str, float]]`**
  Applied in the confidence engine (e.g. `weak_identity_correlation`, `stale_artifact_only`, `missing_parent_child_chain`, `behavioral_only_no_file_artifact`).
- **`evasion_boost: float`**
  Added to the confidence score when evasion indicators are present (Section 4).
- **`action_risk`**
  R1–R4; used by policy (Section 5).
- **`evidence_details`**
  Arbitrary dict for SOC and debugging (e.g. `process_entries`, `behavioral_patterns`, `evasion_findings`).

Base scanner helpers (`_penalize_weak_identity`, `_penalize_stale_artifacts`, `_penalize_missing_process_chain`) standardize common penalty logic across tool scanners.

2.3 Process Tree Construction

Behavioral patterns operate on **process trees**, not flat event lists. `collector/scanner/process_tree.py` builds trees from the event store:

1. **Process events:** `get_process_events()`; for duplicate PIDs, the latest event (by timestamp) is kept.
2. **Network and file events:** `get_network_events()` and `get_file_events()`; grouped by `pid`.
3. **Nodes:** Each unique PID becomes a `ProcessNode` (pid, ppid, name, cmdline, children, network_events, file_events, start_time, username).
4. **Parent-child links:** Nodes are linked by `ppid`; roots are nodes whose `ppid` is 0, 1, or not in the node set.

Trees are returned as a list of root nodes. Scanners that need process-tree context (e.g. BehavioralScanner, correlation) use these structures.

2.4 Behavioral Patterns (BEH-001–BEH-008)

Implemented in `collector/scanner/behavioral_patterns.py`. Each pattern is a function `(ProcessNode tree, thresholds dict) -> PatternMatch` with `pattern_id`, `pattern_name`, `score` in [0, 1], `evidence` dict, and `layers` (which of the five layers the pattern contributes to).

ID	Name	Summary	Layers
BEH-001	Shell fan-out	Many shell children from same parent within a time window	process, behavior
BEH-002	LLM API cadence	Multiple connections to known LLM hosts in a window	network, behavior
BEH-003	Multi-file burst write	Many file creates/modifies across multiple dirs in window	file, behavior
BEH-004	Read-modify-write loop	Interleaved file and network events in short cycles	file, network, behavior

ID	Name	Summary	Layers
BEH-005	Autonomous session duration	Long session with bounded activity gaps	behavior
BEH-006	Config/credential access	Access to sensitive paths (.env, .ssh, .aws, credentials, etc.) plus network	file, network, identity
BEH-007	Git automation	add→commit→push sequences without an editor in tree	process, file, behavior
BEH-008	Process resurrection	Same process name restarted multiple times in short window; upgrades to Class D	process, behavior

LLM host registry: A fixed set of hosts (e.g. `api.openai.com`, `api.anthropic.com`, `localhost:11434`) is used for BEH-002; `behavioral.json` can add `custom_llm_hosts`. Thresholds (window seconds, min counts, etc.) are per-pattern and configurable via `collector/config/behavioral.json`.

2.5 BehavioralScanner Flow

(`collector/scanner/behavioral.py`)

1. **Config:** Load `behavioral.json`; flatten per-pattern thresholds; if `enabled` is false, return a no-detection result.
2. **Trees:** Build process trees from the event store.
3. **Deduplication:** Exclude trees that share any PID with a tool already detected by a named scanner (`exclude_pids`). This avoids double-counting the same process as both a named tool and "Unknown Agent."
4. **Scoring:** For each remaining tree, run `detect_all_patterns(tree, thresholds)`; aggregate scores with pattern-specific weights (BEH-001, BEH-004, BEH-005 get slight boosts). Keep the tree with the highest aggregate score.
5. **Threshold:** If the best aggregate score is below `detection_threshold` (default 0.45), return no detection.

6. **Result:** Set `detected=True`, `tool_name="Unknown Agent"`, `tool_class="C"` (or `"D"` if BEH-008 matched). Build `LayerSignals` from the max per-layer scores of the matching patterns; set `action_risk` from pattern mix (e.g. BEH-006/BEH-008 or BEH-001+BEH-002 → R3). Apply penalties (e.g. `behavioral_only_no_file_artifact` when process+network but no file evidence; weak identity).
7. **Evidence:** Attach `behavioral_patterns`, `root_process`, `tree_pids`, `tree_depth` to `evidence_details`.

2.6 Configuration (behavioral.json)

- **Global:** `enabled`, `detection_threshold`, `custom_llm_hosts`.
- **Per-pattern:** e.g. `BEH-001.shell_fanout_window_seconds`, `shell_fanout_min_children`; `BEH-002.llm_cadence_window_seconds`; etc. Keys are flattened so the scanner sees a single threshold dict.

---

3. Confidence Scoring Model

3.1 Formula (Playbook Appendix B)

Implemented in `collector/engine/confidence.py`.

```
base_score = sum(layer_weight[t] * layer_signal[t])   for t in
{process, file, network, identity, behavior}
penalties  = sum(penalty value for each applicable penalty)
evasion_boost = scan.evasion_boost
final      = clamp(base_score - penalties + evasion_boost, 0, 1)
```

- **Weights** are **per-tool** (and for "Unknown Agent", per `BEHAVIORAL_WEIGHTS`). Defaults: process 0.30, file 0.20, network 0.15, identity 0.15, behavior 0.20. Calibration from lab runs has shifted some tools (e.g. Claude Code, Open Interpreter, GPT-Pilot) to different distributions.
- **Classification:**
  - **High:** score ≥ 0.75
  - **Medium:** 0.45 ≤ score < 0.75
  - **Low:** score < 0.45

3.2 Tool-Specific Weights

The module defines `TOOL_WEIGHTS` with entries for each named tool and for `"Unknown Agent"` (behavior-dominant). Examples:

- **Claude Code:** process 0.32, file 0.23, network 0.12, identity 0.18, behavior 0.15 (lab-run calibration).
- **Ollama:** process/file/network 0.25 each, identity 0.05, behavior 0.20.
- **Open Interpreter:** behavior 0.40, process 0.25, others lower.
- **Unknown Agent (BEHAVIORAL_WEIGHTS):** behavior 0.35, network 0.20, process 0.20, file 0.15, identity 0.10.

3.3 Infrastructure Floor (Class D)

For tools in `INFRASTRUCTURE_FLOOR_TOOLS` (e.g. OpenClaw), if both `signals.process` and `signals.file` are ≥ `INFRASTRUCTURE_FLOOR_THRESHOLD` (0.80), the final score is forced to at least `INFRASTRUCTURE_FLOOR_VALUE` (0.70). This avoids underscoring strong infrastructure evidence when the behavior layer temporarily drops (e.g. small model not generating strong behavior signals).

3.4 Penalties

- Applied by individual scanners via `result.penalties.append((reason, amount))`.
- Common helpers on `BaseScanner`: weak identity, stale artifact only, missing process chain.
- Behavioral scanner adds `behavioral_only_no_file_artifact` (0.15) when there is process and network but no file evidence.

3.5 Calibration and Testing

- Lab-run fixtures under `collector/tests/fixtures/lab_runs/` drive `test_calibration.py`. Changing weights or penalties in `confidence.py` should be validated by re-running calibration so that expected confidence bands (e.g. Medium 0.68–0.74 for certain scenarios) are preserved. CI runs calibration regression on every push/PR.

---

4. Evasion Detection

4.1 Rationale (Playbook Rule C6)

Evasion detection identifies **attempts to suppress AI tool attribution or evade detection**. Findings are treated as a **governance signal**: they **increase** confidence (evasion_boost), on the basis that active hiding of tool use is itself high-signal for policy.

4.2 EvasionScanner Placement

The **EvasionScanner** is a **cross-cutting scanner** that runs after named and behavioral scanners (Stage 1c in `main.run_scan`). It does not target a specific tool; it returns a scan with `tool_name="Evasion Detection"` and `tool_class="X"`. If it finds indicators, that result is appended to `detected_scans` and processed like any other detection (confidence, policy, events). Evasion boost is applied only to **that** scan's confidence (the Evasion Detection result); it is not merged into other tools' scores.

4.3 Vectors (E1–E5)

Implemented in `collector/scanner/evasion.py`.

Vector	Description	Typical boost
E1	Git hook trailer stripping	0.15 (global or repo commit-msg hook that strips Co-Authored-By)
E2	Global template hooks	0.20 (init.templatedir commit-msg hook with stripping patterns)
E3	Force-push after amend/rebase	0.10 (reflog shows amend/rebase then push)
E4	Renamed/wrapped AI binary	0.15 (known AI binary name in cmdline but executable name differs)
E5	Cursor settings	0.10 (git disabled); 0.05 (telemetry off)

Hook stripping detection: Content of commit-msg hooks is matched against regexes (e.g. `co-authored-by`, `grep -v.*co.?authored`, `sed.*co.?authored`, `trailer.*remove`). Global hook path from `git config --global core.hooksPath`; template dir from `init.templatedir`; repo hooks under common search roots (home, Documents, code, projects, etc.), up to `MAX_REPOS_TO_SCAN` (50).

E4: Parses `ps aux` output; for each line containing a known AI binary substring (e.g. `claude`, `cursor`, `ollama`), checks whether the executable stem matches a known name; if not, flags as renamed/wrapped.

E5: Reads Cursor `settings.json` from several known paths (e.g. `~/.cursor/settings.json`, macOS App Support, Windows Roaming); checks `git.enabled === false` and `telemetry.telemetryLevel === "off"`.

4.4 Cap and Signals

- **evasion_boost** is the sum of per-finding boosts, **capped at 0.50**.
- When any finding exists, the scan sets `signals.behavior = 0.8` and `signals.file = 0.6` and fills `evidence_details["evasion_findings"]` with vector, description, and path.

4.5 Operational Notes

- Evasion scan uses filesystem and git config; safe-read limits file size (e.g. 64 KiB). Repo search is limited to avoid excessive I/O. E4 relies on `ps` (Unix) output format.

---

5. Policy Enforcement

5.1 Policy Evaluation (Playbook Section 6.3)

`collector/engine/policy.py` implements **deterministic escalation rules**. Inputs: confidence (float), confidence_class (Low/Medium/High), tool_class (A/B/C/D), sensitivity (Tier0–Tier3), action_risk (R1–R4), optional explicit_deny, network context, containerization, actor_trust_tier (T0–T3), prior_violations.

Output: `PolicyDecision`: `decision_state` (detect | warn | approval_required | block), `rule_id`, `rule_version`, `reason_codes`, `decision_confidence`.

Evaluation order:

1. **Base rules** (`_evaluate_base_rules`): Class D rules (ENFORCE-D01–D03) take precedence, then general rules (ENFORCE-001 through 007 and fallbacks). Examples:
   - Class D + R3+ → block (ENFORCE-D01).
   - Class D + (Medium|High) confidence + R2+ → approval_required (ENFORCE-D02).
   - Class D else → warn (ENFORCE-D03).
   - General: High + R4 → block; Class C + (Medium|High) + R3 → approval_required; Medium + Tier2+ + R3 → approval_required; etc.
2. **Session escalation:** If `prior_violations > 2` and base is warn → approval_required.

3. **Actor escalation:** If actor_trust_tier is T0 and base is detect → warn.
4. **Network overlay:** If `net_ctx` is present and has `unknown_connections > 0`, call `evaluate_network_policy`; if it returns a decision, take the **higher severity** of base and network (overlays only escalate).
5. **Container overlay:** If `is_containerized` is not None and tool_class is C, call `evaluate_container_policy` (ISO-001: Class C must run in container); again take the higher severity.

Network rules (NET-001, NET-002): Class C/D with unknown outbound connections → approval_required (NET-001); with ≥3 unknown connections → block (NET-002, exfiltration risk). Network context is built in main from `scan.evidence_details.get("connections", [])` and an allowlist; if the allowlist is empty, network policy is skipped.

Container rule (ISO-001): Class C tool, not containerized → block. Containerization is determined by `engine/container.py` (Linux: cgroup/mountinfo; macOS: Docker parent chain or docker.sock).

5.2 Enforcement Dispatcher

`collector/enforcement/enforcer.py` maps **policy decision** and **posture** to a **tactic**:

Decision state	Posture	Tactic (summary)
detect, warn	any	log_and_alert (emit events only)
block	passive	log_and_alert
block	audit	simulate (process_kill or network_null_route)
block	active	process_kill or network_block (if conditions met)
approval_required	any	hold_pending_approval (event only)

Active enforcement conditions:

- Not dry_run; posture is active.
- Tool not allow-listed (`PostureManager.is_allow_listed(tool_name)`).
- Allow-list is considered fresh (`PostureManager.is_allow_list_fresh()`); if stale, enforcement is downgraded to audit and a warning is logged.

- `decision_confidence >= auto_enforce_threshold` (from posture manager).
- Rate limiter allows the action (`EnforcementRateLimiter`).

Block + network: If `network_elevated` (rule_id starts with "NET"), the enforcer calls `_network_block`; otherwise it calls `_process_kill` when PIDs are available.

5.3 Posture Manager

`collector/enforcement/posture.py`: Holds **posture** (passive / audit / active), **auto_enforce_threshold**, and **allow_list**. Can be updated from server (e.g. TCP POSTURE_PUSH or heartbeat response). State is persisted under `~/.agentic-gov/posture.json`. **Allow-list staleness:** Time since last sync is tracked; the enforcer uses `is_allow_list_fresh()` to avoid enforcing block when the allow-list may be outdated (e.g. tool just exempted on server but not yet received by agent).

5.4 Process Kill Tactic

`collector/enforcement/process_kill.py`:

- **Target:** Process tree (PID + descendants). Children killed first (leaf to root), then parent. SIGTERM, then after grace period (default 3s) SIGKILL for survivors.
- **Safety:** Optional `expected_pattern`; before killing, cmdline is checked; if the pattern is not in the cmdline, kill is aborted (PID reuse protection).
- **Escalation (enforcer):** If the same tool is killed 3+ times within 300 seconds, the enforcer escalates: attempt to kill parent process; on Linux, try to disable the systemd unit if the process is in a service cgroup; on macOS, try to unload a LaunchAgent/LaunchDaemon plist that references the target executable. Disabled services are recorded in a `DisabledServiceTracker` for potential recovery.

5.5 Network Block Tactic

`collector/enforcement/network_block.py`:

- **macOS:** `pfctl` anchor; block outbound by user (derived from PID). Requires root.
- **Linux:** Prefer cgroup v2 + net_cls + iptables (per-PID). If net_cls is unavailable, fall back to iptables `--uid-owner` (affects all processes of that UID). Requires root/capabilities.
- **Windows:** `netsh advfirewall` rule by executable path. Requires admin.

Unblock routines remove the same rules/anchors. Linux cgroup cleanup moves processes back to root cgroup and removes the detec-block-{pid} cgroup directory.

5.6 Rate Limiting and Results

- **EnforcementRateLimiter:** Limits how many enforcements can occur per minute (default 5). When exceeded, the enforcer returns a result with `rate_limited=True` and does not execute the tactic.
- **EnforcementResult:** tactic, success, detail, tool_name, pid(s), simulated, allow_listed, rate_limited, escalated, escalation_details. All results are appended to the enforcer's `results` list for logging and events.

---

6. End-to-End Pipeline Summary

1. **Bootstrap:** Create `EventStore` (optional `on_alert` in daemon). Start telemetry provider (`auto` / `native` / `polling`); on failure with non-polling, fall back to polling. Load network allowlist if path given.
2. **Telemetry:** If provider has `poll()`, call it once to fill the store for this cycle.
3. **Scans (Stage 1):** Run named scanners (Claude Code, Cursor, Ollama, …) with shared event store; then BehavioralScanner (exclude PIDs already detected); then EvasionScanner; then MCP scanner; then scheduler-artifact enrichment (cron/LaunchAgent). Scheduler can add file-layer signal and create scheduler-only detections.
4. **Correlation (Stage 1f):** `compute_correlation(detected_scans, event_store, extract_pids)` builds a map of tool_name → related tool names (same process tree). Used for event `correlation_context`.
5. **Per-detection (Stage 2):** For each scan in `detected_scans`: compute confidence and class; get PIDs; check containerization; build network context from evidence + allowlist; evaluate policy (base + session + actor + network + container); optionally state-differ (skip if unchanged); build and emit detection.observed, policy.evaluated; if decision is block or approval_required, call enforcer and emit enforcement event (allow_listed / rate_limited / simulated / applied).
6. **Cleared (Stage 3):** If state_differ is used, emit detection.cleared for tools that were previously seen but not in this cycle's detected set (and not in scan_failures).
7. **Teardown:** Provider stopped.

---

7. References and Conventions

- **Playbook:** `playbook/PLAYBOOK-v0.4-*.md` (detection profiles, Appendix B confidence, Section 6.3 rules, Rule C6 evasion).
- **Config:** Collector: `config_loader.py`, `config/collector.json`, `AGENTIC_GOV_*` env. Behavioral: `config/behavioral.json`. Network allowlist: `config/network_allowlist.txt` or path from args.
- **Tests:** `pytest collector/tests/` (e.g. `test_confidence.py`, `test_behavioral_scanner.py`, `test_evasion_scanner.py`, `test_policy.py`, `test_enforcement_e2e.py`, `test_calibration.py`).
- **Versioning:** Playbook and rule version (0.4.0) aligned with EVENT_VERSION and RULE_VERSION in code.

End of report.